

Streaming Machine Learning Framework

Assignment 2 Technical Report

Yen-I Lee · Student ID: 1977309 · Programming for Artificial Intelligence (5004)

1. Overview

This project implements a pure-NumPy streaming machine-learning framework with no dependency on scikit-learn, scipy, or any third-party ML library. All components — scalers, imputers, encoders, decision trees, ensemble classifiers, evaluation metrics, and a training orchestrator — support incremental learning via a unified **partial_fit** interface, enabling the framework to process data in chunks without holding the entire dataset in memory at once.

2. Design Decisions

2.1 partial_fit as Accumulative Rebuild

Rather than a purely online update, **partial_fit** appends each new chunk to an internal buffer and rebuilds the model from scratch on all accumulated data. This guarantees the model always reflects the optimal fit over everything seen so far, at the cost of fit time growing linearly with the number of chunks. For this framework's use case — moderate chunk counts with correctness as the primary goal — this trade-off is intentional and acceptable.

2.2 Chan's Parallel Algorithm for Streaming Statistics

StandardScaler and **Imputer** maintain running mean and variance using Chan's parallel combination algorithm. Rather than iterating over individual rows, each chunk's sufficient statistics (count, mean, M_2) are computed in a single vectorised pass and merged with the running state in $O(d)$ scalar operations. This eliminates Python loops over rows and provides numerical stability even at extreme magnitudes (e.g. $\log=10^4$).

```
# Chan combine: merge chunk stats (n_b, mean_b, M2_b) into running state
delta  = mean_b - self._mean
n_ab   = self._count + n_b_valid
mean_ab = (self._count * self._mean + n_b_valid * mean_b) / n_ab
M2_ab  = self._M2 + M2_b + delta**2 * self._count * n_b_valid / n_ab
```

2.3 Ensemble Methods: Bagging and Random Forest

EnsembleClassifier supports two methods controlled by a single *method* parameter:

- **Bagging** — each tree is trained on a bootstrap sample of the accumulated data using all d features.
- **Random Forest** — same bootstrap sampling, but each tree considers only $\sqrt{d}$ features per split, reducing correlation between trees and improving generalisation.

Predictions are aggregated by majority vote (hard voting). Both methods expose the same **partial_fit** / **predict** / **predict_proba** interface via **RandomForestClassifier**, a thin subclass that sets *max_features='sqrt'* by default.

2.4 Cumulative vs Per-Chunk Metrics

Streaming metric classes (**Accuracy**, **F1Score**, **ConfusionMatrix**) accumulate state across all chunks since the last **reset()**, so **result()** reflects overall model performance — not just the latest batch. Batch convenience functions (*accuracy_score*, *f1_score*, etc.) are provided separately for per-chunk snapshots. This distinction is intentional: long-running cumulative metrics give a stable trend signal; batch functions give immediate feedback.

3. Testing & Edge Cases

The test suite (tests/test_curated.py) contains **39 pytest tests** covering all modules. Tests are run with:

```
pytest tests/test_curated.py -v
```

Module	Key tests & edge cases
StandardScaler	Zero mean after transform; Chan equals batch over two chunks; NaN in chunk does not corrupt state
MinMaxScaler	Output in [0, 1]; zero-range feature no divide-by-zero
Imputer	Mean fill correctness; incremental equals batch (3 chunks); invalid strategy raises ValueError
DecisionTree	Gini / entropy accuracy > 85%; proba sums to 1; depth-0 is leaf; partial_fit accumulates; predict before fit raises RuntimeError
Ensemble	Proba sums to 1; classes preserved across chunks; RF uses sqrt features; streaming accuracy does not regress; invalid method raises ValueError
Pipeline	Fit/predict shape; partial_fit accuracy; classes param forwarded; Imputer fills NaN in pipeline
Metrics	Perfect / worst AUC; cumulative accuracy; streaming ConfusionMatrix matches batch; F1 reset
StreamTrainer	fit_chunk returns dict with accuracy key; log length matches chunk count; score_chunk does not advance log; reset clears log
Integration	pipe.score matches accuracy_score; NaN pipeline no NaN in predictions; streaming ConfusionMatrix shape consistent

Notable edge cases tested: NaN values mid-stream, zero-variance features, all-NaN columns on first chunk, single-sample chunks, classes seen only in one chunk (partial_fit with explicit classes= argument), and prediction before fit (RuntimeError).

4. Benchmark Results

Three models were benchmarked under a simulated streaming scenario: 2,000 samples × 10 features, binary classification, 10 chunks of 200 samples each, 10 estimators for ensemble methods, max depth 5. Per-chunk accuracy and wall-clock fit time were recorded.

Model	Avg Accuracy	Avg Fit Time	Notes
DecisionTreeClassifier (base)	0.8885	3,931 ms	Fastest; lowest memory
EnsembleClassifier — Bagging (n=10)	0.9115	25,159 ms	+3.15 pp over base
RandomForestClassifier (n=10)	0.9200	7,589 ms	Best accuracy; $\sqrt{d}$ features

Key observations. RandomForestClassifier achieves the highest accuracy (+3.15 percentage points over the base tree) with moderate fit time (7,589 ms average). Bagging matches the accuracy gain but is 3.3× slower due to using all features per split. The base DecisionTree is fastest and most memory-efficient, making it the best choice when latency matters more than accuracy. Fit time grows linearly with the number of chunks processed (accumulative rebuild), which is the expected behaviour under the chosen partial_fit strategy.

5. Reflections

What worked well

- Chan's algorithm made StandardScaler numerically stable across 50 chunks at extreme magnitudes ($\text{loc}=10^4$, $\text{tol}=10^{-10}$), a property that naive incremental formulas do not provide.
- The unified partial_fit interface across all transformers and estimators allowed the Pipeline to chain them without any special-casing.
- Separating streaming metrics (cumulative) from batch functions (per-chunk) gave both a stable trend signal and an immediate per-batch view without conflating the two semantics.

Trade-offs and limitations

- **Accumulative rebuild cost.** Fit time grows $O(n \cdot \text{chunks})$, so the framework is not suitable for very long streams or real-time constraints. A true online tree update (e.g. Hoeffding tree) would fix this but adds significant complexity.
- **Median imputation buffers all data.** The median strategy in Imputer concatenates all chunks to recompute the median exactly. For very large streams, a sketch-based quantile estimator (e.g. t-digest) would be more memory-efficient.
- **No feature drift detection.** The framework assumes the data distribution is stationary. Concept drift would silently degrade accuracy without triggering any alert.

Possible improvements

- Replace the accumulative rebuild with a true online tree update algorithm to achieve $O(1)$ per-chunk fit time.
- Add a sliding-window metric mode so recent performance can be monitored independently of the cumulative trend.
- Introduce a drift detector (e.g. ADWIN) that triggers a model reset when the error rate shifts significantly.